

DATA ENGINEERING FUNDAMENTALS

Data Lake • Data Warehouse • Data Mart
Architecture, Workflow, Data Pipelines, Types & Sub-Types

Theory-First Training Reference Guide

Table of Contents

- 1. Data Engineering Fundamentals
- 2. Data Engineering Architecture
- 3. Data Sources and Data Ingestion
- 4. ETL, ELT and Data Transformation
- 5. Data Lake
- 6. Data Lake Architecture and Workflow
- 7. Data Lake Types and Sub-Types
- 8. Data Lake Zones / Layers
- 9. Data Warehouse
- 10. Data Warehouse Architecture and Workflow
- 11. Data Warehouse Types and Models
- 12. Data Mart
- 13. Data Mart Architecture and Workflow
- 14. Data Mart Types and Sub-Types
- 15. Data Lake vs Data Warehouse vs Data Mart
- 16. End-to-End Enterprise Data Pipeline
- 17. Batch vs Streaming Data Engineering
- 18. Data Modeling Concepts
- 19. Data Quality, Governance and Security
- 20. Metadata, Lineage and Observability
- 21. Data Engineering Design Considerations
- 22. Common Patterns and Anti-Patterns
- 23. Important Terminology / Interview Revision
- 24. Final Mental Model

1. Data Engineering Fundamentals

Data Engineering is the discipline of designing, building, operating, and governing systems that collect data from multiple sources, move it reliably, transform it into useful forms, store it appropriately, and make it available for analytics, reporting, machine learning, applications, and decision-making.

1.1 Why Data Engineering Exists

- Operational systems are optimized for transactions, not large-scale analytical processing.
- Organizations have data in databases, files, APIs, SaaS applications, logs, sensors, messages, and third-party systems.
- Raw data is often inconsistent, duplicated, incomplete, differently structured, or difficult to query.
- Analytics and AI require trusted, accessible, historical, and well-organized data.
- Data Engineering creates the reliable path from operational data to analytical value.

1.2 Core Responsibilities

Area	Typical Responsibility
Ingestion	Extract or capture data from source systems.
Storage	Store raw and processed data using suitable storage technologies.
Transformation	Clean, standardize, join, aggregate, enrich and reshape data.
Orchestration	Schedule and coordinate multi-step pipelines.
Modeling	Design schemas, facts, dimensions and analytical structures.
Quality	Validate completeness, correctness, consistency, freshness and uniqueness.
Governance	Manage ownership, metadata, lineage, policies and access.
Security	Protect data using authentication, authorization, encryption and masking.
Serving	Make curated data available to BI, analytics, ML and applications.
Operations	Monitor failures, latency, cost, throughput and pipeline health.

1.3 The Fundamental Data Engineering Lifecycle

1. Discover data sources and business requirements
2. Ingest or capture data
3. Store raw data safely
4. Validate and profile data
5. Transform and enrich data
6. Model curated data
7. Serve data to consumers
8. Monitor, govern and continuously improve

2. Data Engineering Architecture

A data architecture describes how data moves from producers to storage, processing, governance and consumption. A modern architecture is usually composed of multiple logical layers rather than one single database.

2.1 Generic Enterprise Architecture

Sources → Ingestion → Landing/Raw Storage → Processing/Transformation → Curated Storage → Serving/Consumption → BI/ML/Applications

2.2 Major Architectural Components

Layer	Purpose	Examples of Technology Categories
Source	Creates operational or external data	OLTP databases, SaaS, files, APIs, IoT, logs
Ingestion	Moves/captures data	Batch loaders, CDC, message brokers, API ingestion
Storage	Persists data	Object storage, distributed file systems, databases
Processing	Computes transformations	SQL engines, distributed processing, stream processors
Orchestration	Coordinates workflows	Workflow schedulers/orchestrators
Catalog/Governance	Describes and controls data	Catalogs, policy engines, lineage tools
Serving	Provides analytical access	SQL warehouses, marts, semantic layers
Consumption	Uses data	BI, dashboards, notebooks, ML, applications

3. Data Sources and Data Ingestion

3.1 Common Data Source Categories

- Relational databases: Oracle, PostgreSQL, MySQL, SQL Server.
- NoSQL databases: document, key-value, wide-column and graph databases.
- Files: CSV, Excel, JSON, XML, Parquet, Avro, ORC and text files.
- APIs: REST APIs, GraphQL APIs and partner APIs.
- Applications/SaaS: CRM, ERP, HR, finance and marketing systems.
- Events and logs: application events, audit logs, clickstreams and machine logs.
- Streaming/IoT: sensors, devices, telemetry and real-time events.

3.2 Ingestion Patterns

Pattern	Meaning	Typical Use
Full Load	Read the complete source dataset each run.	Small or static tables.
Incremental Load	Read only records changed since the last successful run.	Large transactional tables.
CDC	Capture inserts, updates and deletes from source changes.	Near-real-time database replication.
Event Streaming	Consume events continuously.	Real-time applications and analytics.
File-Based Ingestion	Process newly arrived files.	Data exchanges and batch feeds.
API Polling	Periodically request records from an API.	External systems without CDC.
Replication	Replicate source data to another system.	Operational-to-analytical movement.

4. ETL, ELT and Data Transformation

4.1 ETL

ETL means Extract → Transform → Load. Data is extracted from sources, transformed before loading into the target analytical store, and then made available for consumption.

- Best suited when transformation must happen before data reaches the target or when the target has limited transformation capability.

4.2 ELT

ELT means Extract → Load → Transform. Raw or lightly processed data is first loaded into a scalable analytical platform, then transformed using the target platform's processing engine.

- Common in cloud-scale analytical architectures because storage is inexpensive and compute can be scaled independently.

4.3 Transformation Categories

- Cleansing: null handling, invalid values, duplicate removal.
- Standardization: dates, units, naming conventions and data types.
- Integration: joins across multiple sources.
- Derivation: calculated fields and business metrics.
- Aggregation: daily, monthly, customer or product summaries.
- Enrichment: adding reference, master or external data.
- Conformance: making shared dimensions consistent across subject areas.
- History management: preserving changes over time.

5. Data Lake

A Data Lake is a scalable storage environment designed to hold large volumes of data in its original or near-original form, including structured, semi-structured and unstructured data. The key idea is that data can be stored before all downstream analytical requirements are known.

5.1 Core Characteristics

- Stores raw and processed data at scale.
- Supports structured, semi-structured and unstructured information.
- Separates storage from compute in many modern implementations.
- Supports multiple processing engines and analytical workloads.
- Usually uses object storage or distributed storage as the foundation.
- Can support batch, streaming, analytics, data science and machine learning.
- Requires strong governance; otherwise it can become a data swamp.

5.2 Structured, Semi-Structured and Unstructured Data

Data Type	Characteristics	Examples
Structured	Fixed schema, rows and columns.	Relational tables, transactions.
Semi-Structured	Self-describing or flexible structure.	JSON, XML, Avro.
Unstructured	No fixed tabular schema.	Documents, images, audio, video, text.

5.3 Data Lake Architecture

Sources → Ingestion → Landing/Raw Zone → Cleansed/Standardized Zone → Curated/Trusted Zone → Consumption

5.4 Data Lake Workflow

9. 1. Source systems generate data.
10. 2. Ingestion processes capture files, records or events.
11. 3. Raw data is stored with minimal alteration.
12. 4. Validation checks schema, file integrity and basic quality.
13. 5. Processing cleans and standardizes data.
14. 6. Business transformations create trusted/curated datasets.
15. 7. Catalog and lineage metadata are registered.
16. 8. BI, analytics, data science and ML consumers access the appropriate layer.

6. Data Lake Architecture and Workflow Details

6.1 Typical Lake Zones

Zone	Purpose	Data State
Landing / Ingestion	Temporary arrival location.	Newly received data.
Raw / Bronze	Preserve source data and source history.	Raw or minimally altered.
Cleansed / Silver	Standardize and validate.	Cleaned and conformed.
Curated / Gold	Business-ready analytical datasets.	Trusted and business-defined.
Archive	Long-term retention at lower cost.	Historical/inactive data.

6.2 Lake Processing Flow

Source → Landing → Raw/Bronze → Quality Checks → Silver → Business Rules → Gold → BI/ML/Serving

6.3 Advantages

- Flexible schema evolution
- Supports many data types
- Large-scale storage
- Good fit for data science and ML
- Can retain detailed history
- Supports multiple downstream consumers

6.4 Challenges

- Governance complexity
- Metadata discovery
- Schema inconsistency
- Duplicate data
- Poorly managed lakes can become data swamps
- Potential storage and compute cost without lifecycle management

7. Data Lake Types and Sub-Types

Category	Type / Sub-Type	Description
By storage	Object-storage lake	Built on scalable object storage.
By storage	Distributed-file lake	Built on distributed file-system concepts.
By workload	Batch-oriented lake	Primarily handles periodic loads.
By workload	Streaming lake	Continuously receives event data.
By architecture	Traditional data lake	Raw + processing + analytical layers.
By architecture	Lakehouse	Combines data-lake flexibility with warehouse-style management and SQL analytics.
By deployment	Cloud lake	Runs in public/private cloud storage and compute.
By deployment	On-premises lake	Runs within organizational infrastructure.
By data organization	Zone-based lake	Uses raw, cleansed and curated zones.
By governance	Governed lake	Strong catalog, access, quality and lineage controls.

8. Data Warehouse

A Data Warehouse is a centralized analytical data platform designed to store integrated, historical and structured data for reporting, business intelligence, analytics and decision support.

8.1 Key Characteristics

- Subject-oriented: organized around business subjects such as sales, customers and products.
- Integrated: data from multiple sources is standardized.
- Time-variant: historical values are retained for analysis.
- Non-volatile in the classical sense: analytical data is generally loaded and queried rather than continuously transactionally updated like an OLTP system.
- Optimized for analytical queries rather than high-frequency transactional writes.

8.2 Data Warehouse Architecture

Source Systems → Staging → Integration/Transformation → Enterprise Warehouse → Data Marts/Semantic Layer → BI & Analytics

8.3 Warehouse Workflow

17. Extract data from operational and external sources.
18. Load into staging or landing structures.
19. Profile and validate incoming data.
20. Transform and conform dimensions and measures.
21. Load warehouse fact and dimension structures.
22. Apply history rules where required.
23. Build aggregates, semantic models or marts.
24. Expose trusted datasets to BI and analytical consumers.

9. Data Warehouse Types and Models

9.1 Warehouse Types by Scope

Type	Description
Enterprise Data Warehouse (EDW)	Organization-wide integrated analytical repository.
Operational Data Store (ODS)	Integrated store often used for near-current operational reporting; not a replacement for a full warehouse.
Cloud Data Warehouse	Warehouse delivered on cloud infrastructure with elastic storage/compute.
Departmental Warehouse	Warehouse focused on a specific organizational area.

9.2 Dimensional Modeling

Dimensional modeling organizes analytical data into fact tables containing measurable business events and dimension tables containing descriptive context.

Component	Meaning	Examples
Fact	Business event or measurable activity.	Sales transaction, shipment, claim.
Dimension	Descriptive context used to analyze facts.	Customer, product, date, location.
Measure	Numeric value stored or calculated.	Revenue, quantity, cost.
Grain	Exact meaning of one fact row.	One product line per invoice.
Surrogate Key	Warehouse-generated dimension identifier.	CustomerKey = 10025.

9.3 Star Schema

A central fact table connects directly to denormalized dimension tables. It is simple for BI queries and commonly used in analytical modeling.

9.4 Snowflake Schema

A snowflake schema normalizes some dimensions into additional related tables. It can reduce duplication but increases joins and model complexity.

9.5 Fact Table Sub-Types

- Transaction fact: one row per business event.
- Periodic snapshot fact: one row per entity at a regular time interval.
- Accumulating snapshot fact: tracks milestones through a business process.
- Factless fact: records that an event or relationship occurred without a numeric measure.

9.6 Dimension Sub-Types

- Conformed dimension
- Role-playing dimension
- Degenerate dimension
- Junk dimension
- Slowly Changing Dimension (SCD)
- Mini-dimension

10. Data Mart

A Data Mart is a focused analytical data store designed for a particular department, business function, subject area or user group. It can be sourced from an enterprise warehouse, directly from operational sources, or from a lake/lakehouse.

10.1 Data Mart Architecture

Source Systems → Integration/Warehouse or Lake → Subject-Area Transformation → Data Mart → BI / Departmental Analytics

10.2 Data Mart Workflow

25. Identify a department or subject area.
26. Identify required dimensions, facts and metrics.
27. Source trusted data from the enterprise platform or source systems.
28. Transform data according to departmental requirements.
29. Load the mart using an agreed grain and business definitions.
30. Apply access controls and refresh schedules.
31. Expose dashboards, reports and analytical datasets.

10.3 Data Mart Types

Type	Description	Example
Dependent Mart	Built from an enterprise warehouse.	Finance mart sourced from EDW.
Independent Mart	Built directly from operational/external sources.	Marketing mart built independently.
Hybrid Mart	Combines warehouse and other sources.	Sales mart combining EDW and external market data.
Virtual Mart	Logical views/semantic structures rather than a separate physical store.	Subject-area views over warehouse tables.
Departmental Mart	Designed around a department.	HR, Finance, Sales.
Subject-Area Mart	Designed around a business subject.	Customer, Product, Orders.

11. Data Lake vs Data Warehouse vs Data Mart

Dimension	Data Lake	Data Warehouse	Data Mart
Primary purpose	Broad-scale storage and multi-workload processing.	Integrated enterprise analytics.	Focused departmental/subject analytics.
Typical data	Structured + semi-structured + unstructured.	Primarily structured/curated.	Primarily structured/curated.
Schema approach	Often schema-on-read/flexible.	Schema-on-write/controlled analytical model.	Controlled subject-area schema.
Users	Data engineers, scientists, analysts, ML teams.	BI, analysts, data engineers, business users.	Departmental analysts and business users.
Scope	Enterprise/platform-wide.	Enterprise-wide or large analytical domain.	Department/subject area.
Granularity	Can retain raw detailed data.	Curated analytical detail and history.	Focused detail/aggregates.
Typical query experience	Depends on processing/query layer.	Optimized analytical SQL.	Highly focused analytical queries.
Governance need	Very high.	Very high.	High but narrower in scope.

12. End-to-End Enterprise Data Pipeline

An enterprise pipeline connects operational systems to analytical consumers through controlled stages. The exact implementation varies, but the logical flow remains similar.

12.1 Reference Pipeline

SOURCE → INGEST → LAND → RAW → VALIDATE → CLEANSE → TRANSFORM → CURATE → MODEL → SERVE → CONSUME → MONITOR

12.2 Pipeline Stages

Stage	Key Questions
Source	Where is the data generated? Who owns it?
Ingestion	How is it captured? Full, incremental, CDC or streaming?
Landing	Was the incoming payload received successfully?
Raw	Can the original data be replayed or audited?
Validation	Are schema, completeness and basic quality acceptable?
Transformation	What business rules are applied?
Curated	Is the dataset trusted and reusable?
Modeling	What facts, dimensions and grain are required?
Serving	How will consumers access the data?
Consumption	Who uses it and for what decision?
Monitoring	Is it fresh, complete, accurate and within SLA?

12.3 Pipeline Reliability Concepts

- Idempotency: rerunning a pipeline should not unintentionally duplicate results.
- Checkpointing: remember progress so processing can resume.
- Retry: recover from transient failures.
- Dead-letter handling: isolate records/events that cannot be processed.
- Backfill: process historical data for a missing or corrected period.
- Replay: reprocess source events from retained history.
- Dependency management: execute tasks only when prerequisites succeed.
- SLA/SLO: define expected completion and reliability targets.

13. Batch vs Streaming Data Engineering

Characteristic	Batch	Streaming
Processing	Runs periodically.	Continuous or near-continuous.
Latency	Minutes, hours or days.	Milliseconds to seconds/minutes depending on design.
Typical use	Daily reports, billing, historical loads.	Fraud alerts, telemetry, real-time monitoring.
Complexity	Generally simpler operational model.	Higher operational and state-management complexity.
Correctness concerns	Late-arriving files/records.	Ordering, duplicates, late events and state.
Typical trigger	Schedule or file arrival.	Event/message arrival.

13.1 Streaming Concepts

- Event, producer and consumer
- Topic/stream
- Partition

- Offset
- Consumer group
- Event time vs processing time
- Windowing
- Watermarks
- Exactly-once / at-least-once / at-most-once delivery semantics

14. Data Modeling Concepts

14.1 Three Broad Modeling Levels

Level	Purpose
Conceptual Model	High-level business entities and relationships.
Logical Model	Detailed attributes, keys and relationships independent of physical storage.
Physical Model	Actual tables, columns, indexes, partitions, file formats and storage structures.

14.2 OLTP vs OLAP

Characteristic	OLTP	OLAP
Purpose	Run business transactions.	Analyze business data.
Workload	Many short reads/writes.	Fewer but complex analytical queries.
Data	Current operational state.	Historical and integrated data.
Modeling	Often normalized.	Often dimensional/analytical.
Optimization	Transaction throughput and consistency.	Scan, aggregation and analytical performance.

15. Data Quality, Governance and Security

15.1 Data Quality Dimensions

- Accuracy
- Completeness
- Consistency
- Timeliness/Freshness
- Uniqueness
- Validity
- Integrity

15.2 Governance Domains

- Data ownership and stewardship
- Business glossary
- Metadata management
- Data catalog
- Lineage
- Classification
- Retention

- Access policies
- Compliance
- Master/reference data management

15.3 Security Controls

- Authentication
- Authorization/RBAC
- Least privilege
- Encryption at rest
- Encryption in transit
- Column/row-level security
- Data masking/tokenization
- Audit logging
- Secrets management

16. Metadata, Lineage and Observability

16.1 Metadata Types

Type	Examples
Technical Metadata	Table, column, datatype, partition, file format.
Business Metadata	Business definition, owner, glossary term, KPI definition.
Operational Metadata	Load time, row count, status, duration, failure reason.
Lineage Metadata	Source-to-target relationships and transformations.

16.2 Data Observability

Data observability means continuously understanding the health of data and pipelines. A useful framework includes freshness, volume, distribution, schema and lineage/quality signals.

- Freshness checks
- Row-count/volume checks
- Schema drift detection
- Null and uniqueness checks
- Distribution/anomaly checks
- Pipeline failure alerts
- Source-to-target reconciliation

17. Data Engineering Design Considerations

- Volume: How much data is generated now and how fast is it growing?
- Velocity: How quickly must data become available?
- Variety: What formats and data types exist?
- Veracity: How reliable and consistent is the source data?
- Latency: What is the maximum acceptable delay?
- Retention: How long must data be stored?
- Recovery: How will failed pipelines and corrupted data be recovered?
- Cost: What are the storage, compute, network and operational costs?

- Security: Which users and systems can access which data?
- Governance: Who owns the data and how is its meaning controlled?
- Scalability: Can the architecture handle future growth?
- Maintainability: Can engineers understand and change the pipeline safely?

18. Common Architecture Patterns and Anti-Patterns

18.1 Common Patterns

- Raw → Cleansed → Curated zone pattern
- Medallion architecture: Bronze → Silver → Gold
- Lambda-style separation of batch and streaming paths
- Kappa-style stream-centric processing
- Warehouse + dependent marts
- Lake + warehouse/lakehouse serving architecture
- CDC-based incremental ingestion
- Metadata-driven pipelines

18.2 Common Anti-Patterns

- Loading everything into one uncontrolled storage area.
- No data ownership or business definitions.
- Pipelines that cannot be safely rerun.
- Hard-coded business logic scattered across scripts.
- No schema evolution strategy.
- No data quality checks.
- No monitoring or alerting.
- Building many independent marts with conflicting KPI definitions.
- Keeping raw data forever without retention/lifecycle policy.

19. Important Data Engineering Terminology

Term	Meaning
Data Pipeline	Automated flow that moves and transforms data between systems.
Data Platform	Collection of storage, compute, ingestion, governance and serving capabilities.
Data Lake	Scalable repository for raw and processed data of many types.
Data Warehouse	Integrated analytical repository optimized for BI and analytics.
Data Mart	Focused analytical store for a department or subject area.
Data Lakehouse	Architecture combining flexible lake storage with stronger table management and analytical capabilities.
Schema-on-Write	Schema is defined/enforced before data is stored for analytical use.
Schema-on-Read	Structure is interpreted when data is read/queried.
CDC	Change Data Capture; captures inserts, updates and deletes from a source.
SCD	Slowly Changing Dimension; techniques for managing dimension history.
Partitioning	Dividing data into logical storage/query units using partition keys.
Bucketing/Clustering	Organizing data to improve access patterns depending on the

	platform.
Compaction	Combining many small files/segments into fewer larger ones.
Data Lineage	Record of where data originated and how it moved/transformed.
Data Catalog	Searchable inventory describing available data assets.
Data Contract	Explicit agreement about schema, semantics, ownership and expectations between producers and consumers.
Data Mesh	Organizational/architectural approach emphasizing domain ownership and data as a product.
Data Product	A governed, discoverable, reusable dataset/service designed for consumer needs.
Orchestration	Coordinating task dependencies, schedules, retries and workflow execution.
Backfill	Reprocessing historical data for a missing/corrected period.
Idempotency	Property where repeated execution produces the same intended result without unintended duplication.
Data Drift	Change in data patterns or distributions over time.
Schema Drift	Unexpected change in structure, columns or data types.
Data SLA	Expected delivery/availability commitment for data.
Semantic Layer	Business-oriented layer that standardizes metrics and analytical meaning.

20. Knowledge Check: Questions Participants Should Be Able to Answer

32. What is Data Engineering and how is it different from Data Analysis and Data Science?
33. What is the difference between a Data Lake, Data Warehouse and Data Mart?
34. When would you choose a Data Lake instead of a Data Warehouse?
35. What is ETL vs ELT?
36. What is the difference between full load, incremental load and CDC?
37. What are Bronze, Silver and Gold layers?
38. What is the purpose of a staging area?
39. What is a fact table? What is a dimension table?
40. What does grain mean in dimensional modeling?
41. What is the difference between a star schema and a snowflake schema?
42. What are transaction, periodic snapshot and accumulating snapshot facts?
43. What is an SCD and why is it needed?
44. What is a dependent vs independent data mart?
45. What is the difference between batch and streaming pipelines?
46. What are event time, processing time and watermark?
47. What is data lineage and why does it matter?
48. What is schema drift?
49. What does idempotency mean in a pipeline?
50. What are data quality dimensions?
51. What is the difference between OLTP and OLAP?
52. What is a data lakehouse?
53. What is a data swamp and how can it be prevented?
54. What are the major layers of an enterprise data architecture?
55. How would you design an end-to-end pipeline from an operational database to a BI dashboard?
56. What monitoring should be applied to a production data pipeline?

21. Final Mental Model

Participants should leave with the following mental model:

- DATA ENGINEERING = BUILDING THE RELIABLE DATA HIGHWAY.
- DATA LAKE = STORE BROAD DATA, INCLUDING RAW DATA, AT SCALE.
- DATA WAREHOUSE = INTEGRATE AND MODEL TRUSTED DATA FOR ANALYTICS.
- DATA MART = DELIVER A FOCUSED ANALYTICAL VIEW TO A DEPARTMENT OR SUBJECT AREA.
- PIPELINE = MOVE + VALIDATE + TRANSFORM + MODEL + SERVE + MONITOR.
- GOVERNANCE = KNOW WHAT THE DATA MEANS, WHO OWNS IT, WHERE IT CAME FROM, WHO CAN USE IT, AND HOW LONG IT SHOULD BE RETAINED.
- QUALITY = ENSURE DATA IS ACCURATE, COMPLETE, CONSISTENT, UNIQUE, VALID AND FRESH.
- RELIABILITY = DESIGN FOR RETRIES, REPLAY, BACKFILL, RECOVERY AND SAFE RERUNS.

Appendix A — Quick Comparison Cheat Sheet

Question	Data Lake	Data Warehouse	Data Mart
What is it?	Broad storage and processing platform.	Integrated analytical repository.	Focused analytical repository.
Main goal?	Flexibility and scale.	Trusted enterprise analytics.	Department/subject analytics.
Data before transformation?	Commonly retained.	Usually transformed/curated.	Usually curated.
Schema flexibility?	High.	Controlled.	Controlled.
Typical consumer?	Engineers, scientists, analysts.	BI and analytics users.	Departmental users.
Can it contain history?	Yes.	Yes.	Yes, as designed.
Main risk?	Poor governance/data swamp.	Complex integration/model management.	Siloed/conflicting definitions if unmanaged.